Functions

Introduction to Computer Programming

Chandra Gummaluru

chandra-gummaluru.github.io

Built-in functions only go so far. We can **define** our own.

one answer at a time

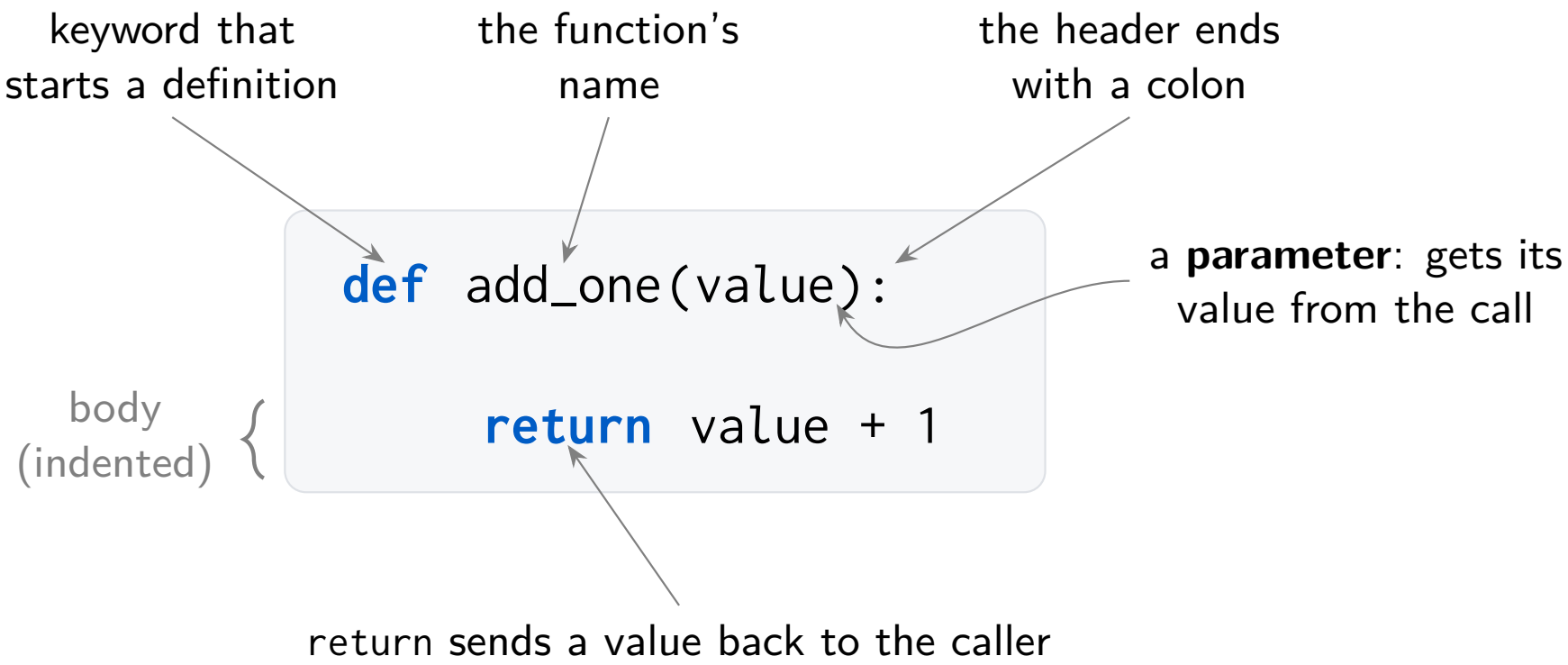
```
>>> 20 * 12 / 2
120.0
>>> 7 * 3 / 2
10.5
>>> 9 * 4 / 2
18.0
```

teach it once, use it forever

```
>>> def area(base, height):
...     return base * height / 2
>>> area(20, 12)
120.0
>>> area(7, 3)
10.5
```

This way, instead of typing a calculation every time, we teach Python how to do it once.

A **function definition** has a **header** and an indented **body**.



Parameters appear in the function definition. **Arguments** appear in the function call.

definition

```
def add_one(value):
    return value + 1
```

calls

```
>>> add_one(9)
10
>>> result = add_one(4)
>>> result
5
```

Here, `value` is the parameter, and `9` and `4` are arguments.

In the call `area(7, 3)`, what are `7` and `3` called?

A parameters

✓ arguments

C return values

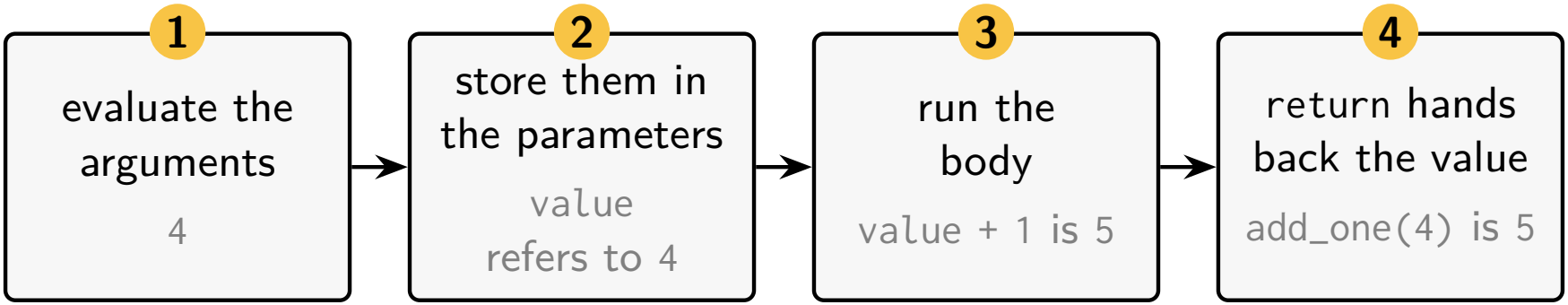
D type contracts

How many arguments must we provide when we call this function?

```
1  def average(a, b, c):
2      return (a + b + c) / 3
```

Answer: `3`

When `Python` runs a **function call** such as `result = add_one(4)`, it goes through four steps.



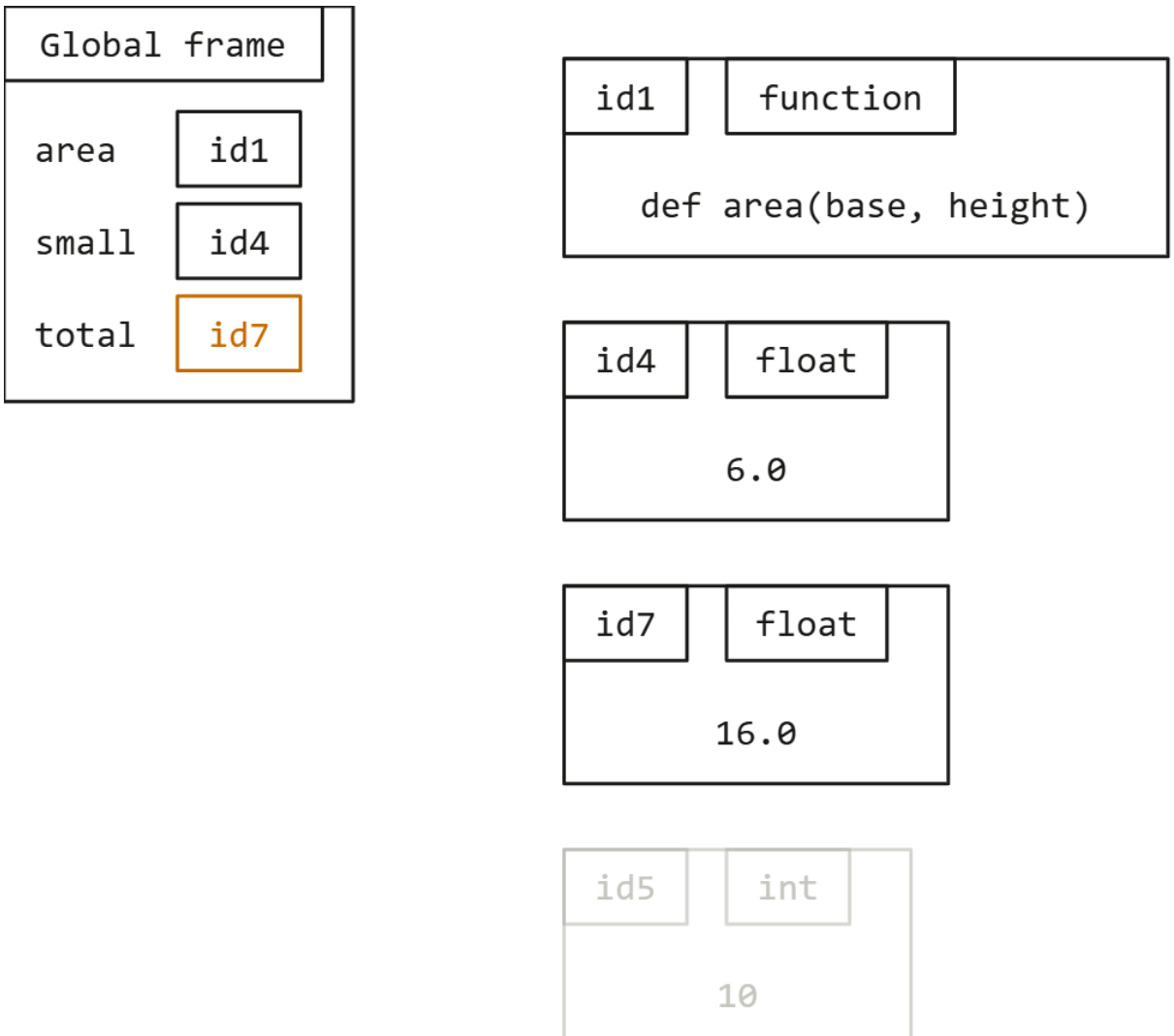
Arguments are evaluated left to right. A `return` statement ends the function right away, and the call produces the returned value, so here `result` refers to `5`.

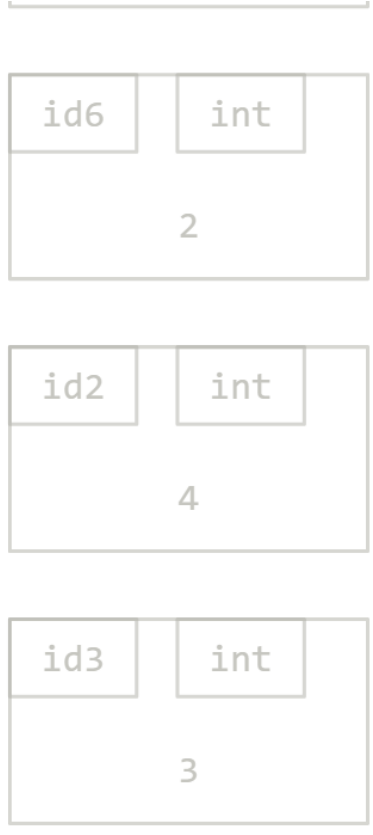
area.py

Program

```
1  def area(base: float, height: float) -> float:
2      return base * height / 2
3
4
5  small = area(4, 3)
6  total = area(10, 2) + small
```

Memory model (step 5)


Global frame
area
id1
small
id4
total
id7
id1
function
def area(base, height)
id4
float
6.0
id7
float
16.0
id5
int
10


id6
int
2
id2
int
4
id3
int
3

documenting functions

A function typed into the shell disappears when the shell restarts, so we write functions in a **.py file** instead.

the shell

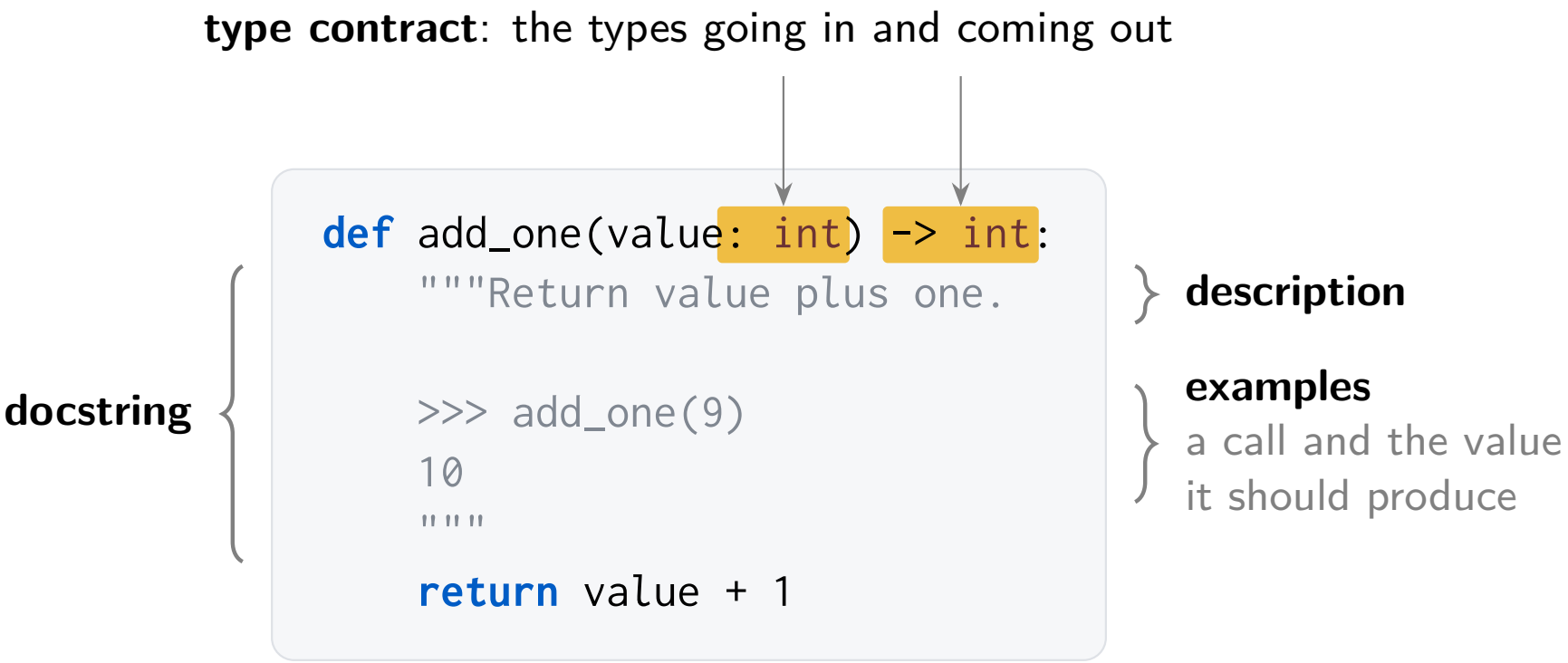
runs one line at a time, and forgets it all when it restarts

a .py file

is saved, so you can edit it and run it again

1. write it in a .py file → 2. click **Run** → 3. call it in the shell

We **document** a function so that others (and future you) know how to use it without reading its body.



The **docstring** is the triple-quoted text right after the header; `help(add_one)` shows it.

double.ipynb

Function `double` takes a number and returns twice its value.
Finish the examples by writing the returned value. Then, write a `return` statement to complete the function definition.

In [ ]

```
def double(num: float) -> float:
    """Return twice the value of num.

    >>> double(7.0)
    14.0
    >>> double(5.7)
    11.4
    """
    return num * 2
```

In [2]

```
double(7.0)
```

```
14.0
```

our_maximum.ipynb

Function `our_maximum` takes two numbers and returns the larger of the two.
Finish the examples by writing the returned value. Then, complete the function definition.

In [ ]

```
def our_maximum(num1: float, num2: float) -> float:
    """Return the larger of num1 and num2.

    >>> our_maximum(1.5, 2.5)
    2.5
    >>> our_maximum(4.0, 3.7)
    4.0
    """
    return max(num1, num2)
```

In [2]

```
our_maximum(1.5, 2.5)
```

```
2.5
```

A function body without a `return` statement still produces a value: `None`. The shell doesn't display `None`, so it looks like nothing happened.

```
def add_two(value: int) -> int:
    value + 2
```

```
>>> add_two(3)
>>> type(add_two(3))
<class 'NoneType'>
```

`value + 2` is evaluated and then thrown away. To send a value back, the body needs a `return`.

Function `max_of_min` takes `num1`, `num2`, `value1` and `value2`, determines the minimum of `num1` and `num2`, the minimum of `value1` and `value2`, and returns the maximum of those minimums.

```
max_of_min(4.0, 3.7, 6.0, 3.5)
```

Answer: `3.7`

```
                    max_of_min(1.0, 1.7,
4.5, 3.0)
```

Answer: `3`

max_of_min.ipynb

Function `max_of_min` takes four values, `num1`, `num2`, `value1`, and `value2`, determines the minimum of `num1` and `num2`, the minimum of `value1` and `value2`, and returns the maximum of those minimums.

Use your answers for `max_of_min(4.0, 3.7, 6.0, 3.5)` and `max_of_min(1.0, 1.7, 4.5, 3.0)` to fill in two examples in the docstring. Then, complete the function definition. If you like, you can use several statements.

In [ ]

```
def max_of_min(num1: float, num2: float, value1: float, value2: float) -> float:
    """Return the maximum of the minimums of the pairs num1 and num2,
    and value1 and value2.

    >>> max_of_min(4.0, 3.7, 6.0, 3.5)
    3.7
    >>> max_of_min(1.0, 1.7, 4.5, 3.0)
    3.0
    """
    min1 = min(num1, num2)
    min2 = min(value1, value2)
    return max(min1, min2)
```

In [2]

```
max_of_min(4.0, 3.7, 6.0, 3.5)
```

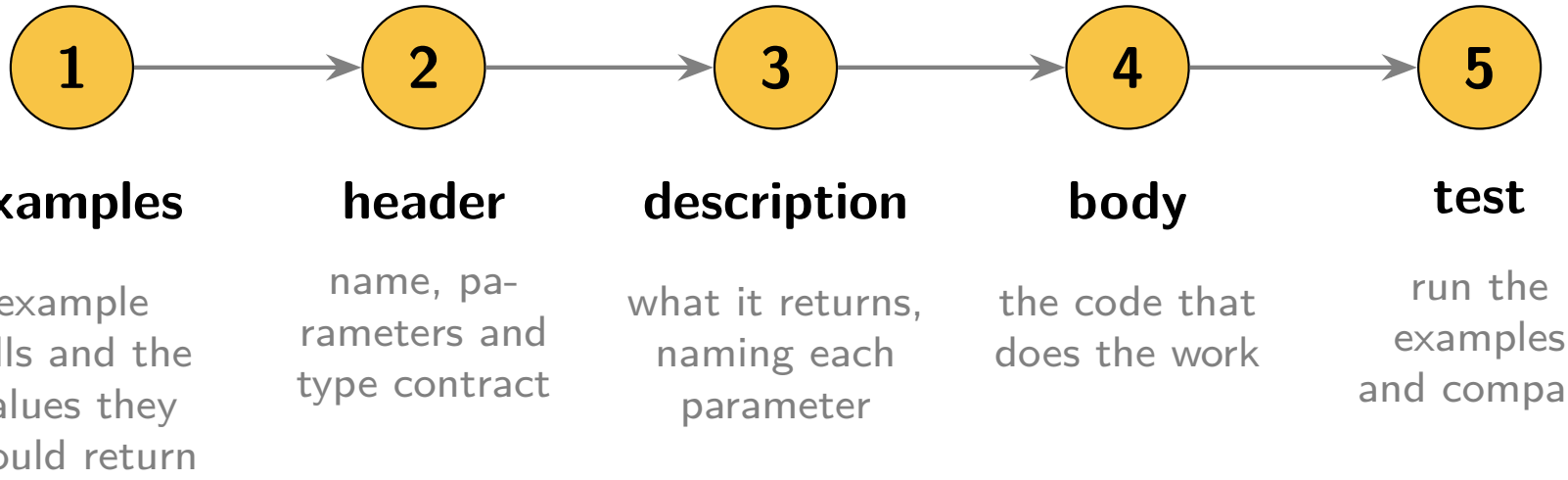
```
3.7
```

the function design recipe

The **Function Design Recipe** is a step-by-step process for writing a function. Like a cooking recipe, it isn't the only way, but following it gives reliable results.

it documents	it guides	it checks
others can see what the function does without reading the body	each step answers one question, so you never start from a blank file	the examples you write first become the tests you run last

The recipe has five steps, done in order.



Examples. Pick a name, then write a few calls and the values you expect, inside a triple-quoted string.

```
    """
    >>> is_even(2)
    True
    >>> is_even(17)
    False
    """
```

pick examples with different answers: one `True`, one `False`

Header. Add the header above the examples, with a meaningful parameter name and the type contract.

```
def is_even(value: int) -> bool:
    """
    >>> is_even(2)
    True
    >>> is_even(17)
    False
    """
```

an `int` goes in, and a `bool` comes out

Description. Before the examples, say what the function returns. Start with a verb and mention each parameter by name.

```
def is_even(value: int) -> bool:
    """Return True if and only if value is even.

    >>> is_even(2)
    True
    >>> is_even(17)
    False
    """
```

the description says *what* the function does, not *how*

Body. Now that we've decided *what* the function does, work out *how*. Look back at how you worked out the examples.

```
def is_even(value: int) -> bool:
    """Return True if and only if value is even.

    >>> is_even(2)
    True
    >>> is_even(17)
    False
    """
    return value % 2 == 0
```

we knew `is_even(2)` is `True` because `2` divides evenly by `2`

A **precondition** is an assumption about the arguments, written in the docstring.

```
def split_bill(total_cost: float, n: int) -> float:
    """Return each person's share when total_cost
    is split evenly between n people.

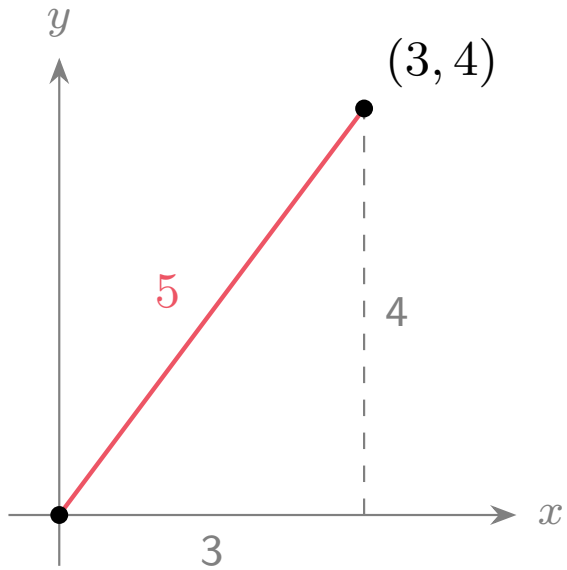
    Precondition: n > 0

    >>> split_bill(21.0, 4)
    5.25
    """
    return total_cost / n
```

the author	the caller
writes the assumption down	makes sure it holds

If a call breaks the precondition, like `split_bill(10.0, 0)`, the function doesn't promise anything.

The **distance** of a point (x, y) from the origin $(0, 0)$ is given by a formula. The math isn't the point here; we just need to turn it into code.



$$\sqrt{x^2 + y^2}$$

in Python, a square root is `** 0.5`

Next, we follow the recipe to write `get_distance` in a file, `distance.py`.

distance.py

Program

```
def get_distance(x: float, y: float) -> float:
    """Return the distance of the point (x, y) from the origin (0, 0).

    >>> get_distance(3.0, 4.0)
    5.0
    >>> get_distance(0.0, 0.0)
    0.0
    """
    return (x ** 2 + y ** 2) ** 0.5
```

Shell

```
=== RUN distance.py ===
```

```
>>> get_distance(3.0, 4.0)
5.0
>>> get_distance(0.0, 0.0)
0.0
```

```
>>> get_distance(1.0, 1.0)
1.4142135623730951
```

Memory model (step 1)

How many arguments must we provide when calling `get_distance`?

A 0

B 1

2

D 3

What is the return type of `get_distance`?

A `int`

✓ `float`

C `str`

D `bool`

repeat_word.ipynb

Following the Function Design Recipe, write a function that satisfies this description:

This function returns a string containing a given word repeated a given number of times. For example, someone should be able to call the function to repeat `"Marcia"` three times and the function should return `"MarciaMarciaMarcia"`, or call the function to repeat `"Buffalo"` eight times and have it return `"BuffaloBuffaloBuffaloBuffaloBuffaloBuffaloBuffaloBuffalo"`.

Remember the steps: **1. Examples, 2. Header, 3. Description, 4. Body, 5. Test**

In [ ]

```
def repeat_word(word: str, num_times: int) -> str:
    """Return word repeated num_times times.

    >>> repeat_word('Marcia', 3)
    'MarciaMarciaMarcia'
    >>> repeat_word('Buffalo', 8)
    'BuffaloBuffaloBuffaloBuffaloBuffaloBuffaloBuffaloBuffalo'
    """
    return word * num_times
```

In [2]
```
repeat_word('Marcia', 3)
```

```
'MarciaMarciaMarcia'
```

In [3]
```
repeat_word('Buffalo', 8)
```

```
'BuffaloBuffaloBuffaloBuffaloBuffaloBuffaloBuffaloBuffalo'
```

Which type contract fits `number_of_cents`?

```
def number_of_cents(change: ?) -> ?:
    ...
    >>> number_of_cents(1.25)
    25
    >>> number_of_cents(20.00)
    0
    ...
    dollars = change // 1
    cents = (change - dollars) * 100
    return round(cents)
```

- ✓ `(change: float) -> int`
- B `(change: int) -> float`
- C `(change: float) -> float`
- D `(change: str) -> int`

number_of_cents.ipynb

Consider the function `number_of_cents` below.

(a) Add the missing *type contract* (parameter and return types) to the function header.

(b) The description of this function is poor: *The number of cents left when you take away all the dollars from an amount of money.* In the space provided in the docstring, write a better function description.

In [ ]

```
def number_of_cents(change: float) -> int:
    """Return the number of cents left in change after all of the whole
    dollars have been taken out.

    >>> number_of_cents(1.25)
    25
    >>> number_of_cents(20.00)
    0
    """
    dollars = change // 1              # alternate: dollars = int(change)
    cents = (change - dollars) * 100
    return round(cents)
```

In [2]

```
number_of_cents(1.25)
```

```
25
```

calculate_tax.ipynb

Following the Function Design Recipe, write the following function.

Function name: (parameter types) → return type	Description
`(float, float) float`	The first parameter represents a bill and the second represents a tax rate (a number between 0.0 and 1.0 inclusive). Return the amount of tax to be paid on this bill.

In [ ]

```python
def calculate_tax(bill: float, tax_rate: float) -> float:
    """Return the amount of tax to be paid on bill for a given tax_rate.

    Precondition: 0.0 <= tax_rate <= 1.0

    >>> calculate_tax(2.00, 0.13)
    0.26
    >>> calculate_tax(30.00, 0.05)
    1.5
    """
    return bill * tax_rate
```

In [2]

```
calculate_tax(30.00, 0.05)
```

```
1.5
```

testing



Test. Run the file, call the function on each docstring example in the shell, and compare.

docstring says

```
>>> is_even(2)
True
>>> is_even(17)
False
```

same?

shell gives

```
>>> is_even(2)
True
>>> is_even(17)
False
```

Test every example, since a body can pass one and fail another. Then try tricky cases you didn't write down, like `is_even(0)` or `is_even(-3)`.

testing_repeat_word.ipynb

Here are three different implementations of `repeat_word`. Run each one, call it with **both** docstring examples, and record whether it returns the expected result. If not, what happens instead?

Version 1

In [ ]

```python
def repeat_word_v1(word: str, num_times: int) -> str:
    """Return word repeated num_times times.

    >>> repeat_word_v1('Marcia', 3)
    'MarciaMarciaMarcia'
    >>> repeat_word_v1('Buffalo', 8)
    'BuffaloBuffaloBuffaloBuffaloBuffaloBuffaloBuffaloBuffalo'
    """
    word = 'Marcia'
    num_times = 3
    return word * num_times
```

In [2]
```
repeat_word_v1('Marcia', 3)
```

```
'MarciaMarciaMarcia'
```

In [3]
```
repeat_word_v1('Buffalo', 8)
```

```
'MarciaMarciaMarcia'
```

Version 1 ignores its arguments, so it passes the first example but returns `'MarciaMarciaMarcia'` for the second.

Version 2

In [ ]

```python
def repeat_word_v2(word: str, num_times: int) -> str:
    """Return word repeated num_times times.

    >>> repeat_word_v2('Marcia', 3)
    'MarciaMarciaMarcia'
    >>> repeat_word_v2('Buffalo', 8)
    'BuffaloBuffaloBuffaloBuffaloBuffaloBuffaloBuffaloBuffalo'
    """
    result = word * num_times
    return result
```

In [5]

```python
repeat_word_v2('Marcia', 3)
```

```
'MarciaMarciaMarcia'
```

In [6]

```python
repeat_word_v2('Buffalo', 8)
```

```
'BuffaloBuffaloBuffaloBuffaloBuffaloBuffaloBuffaloBuffalo'
```

Version 2 passes both examples.

Version 3

In [ ]

```
def repeat_word_v3(word: str, num_times: int) -> str:
    """Return word repeated num_times times.

    >>> repeat_word_v3('Marcia', 3)
    'MarciaMarciaMarcia'
    >>> repeat_word_v3('Buffalo', 8)
    'BuffaloBuffaloBuffaloBuffaloBuffaloBuffaloBuffaloBuffalo'
    """
    word * num_times
```

In [ ]

```
repeat_word_v3('Marcia', 3)
```

In [ ]

```
repeat_word_v3('Buffalo', 8)
```

Version 3 forgets the `return`, so both calls return `None` and nothing is shown. It fails both examples.

Which docstring examples does version 1 of `repeat_word` pass?

```
def repeat_word_v1(word: str, num_times: int) -> str:
    """Return word repeated num_times times.

    >>> repeat_word_v1('Marcia', 3)
    'MarciaMarciaMarcia'
    >>> repeat_word_v1('Buffalo', 8)
    'BuffaloBuffaloBuffaloBuffaloBuffaloBuffaloBuffaloBuffalo'
    """
    word = 'Marcia'
    num_times = 3
    return word * num_times
```

A. both examples

✓ only `('Marcia', 3)`

C. only `('Buffalo', 8)`

D. neither example

Which docstring examples does version 2 of `repeat_word` pass?

```
def repeat_word_v2(word: str, num_times: int) -> str:
    """Return word repeated num_times times.

    >>> repeat_word_v2('Marcia', 3)
    'MarciaMarciaMarcia'
    >>> repeat_word_v2('Buffalo', 8)
    'BuffaloBuffaloBuffaloBuffaloBuffaloBuffaloBuffaloBuffalo'
    """
    result = word * num_times
    return result
```

- ✓ both examples
- B only `('Marcia', 3)`
- C only `Buffalo', 8)`
- D neither example

Which docstring examples does version 3 of `repeat_word` pass?

```
1  def repeat_word_v3(word: str, num_times: int) -> str:
2      """Return word repeated num_times times.
3
4      >>> repeat_word_v3('Marcia', 3)
5      'MarciaMarciaMarcia'
6      >>> repeat_word_v3('Buffalo', 8)
7      'BuffaloBuffaloBuffaloBuffaloBuffaloBuffaloBuffaloBuffalo'
8      """
9      word * num_times
```

A both examples

B only `Marcia', 3)`

C only `Buffalo', 8)`

✓ neither example

function reuse

A function can call another function we wrote. This lets a **helper function** do part of the work.

```
def square(num: float) -> float:
    """Return num multiplied by itself."""
    return num * num

def get_distance(x: float, y: float) -> float:
    """Return the distance of (x, y) from (0, 0)."""
    return (square(x) + square(y)) ** 0.5
```

`get_distance` only needs to know what `square` returns, not how it works.

helper.py

Program

```
def calculate_tax(bill: float, tax_rate: float) -> float:
    """Return the amount of tax to be paid on bill for a given tax_rate.

    Precondition: 0.0 <= tax_rate <= 1.0

    >>> calculate_tax(2.00, 0.13)
    0.26
    >>> calculate_tax(30.00, 0.05)
    1.5
    """
    return bill * tax_rate


def calculate_bill(bill: float, tax_rate: float) -> float:
    """Return the total amount to pay for bill, including tax at the
    given tax_rate.

    Precondition: 0.0 <= tax_rate <= 1.0

```

```
20      >>> calculate_bill(2.00, 0.13)
21      2.26
22      >>> calculate_bill(30.00, 0.05)
23      31.5
24      """
25      return bill + calculate_tax(bill, tax_rate)
```

Shell

```
=== RUN helper.py ===
```

```
>>> calculate_tax(2.00, 0.13)
0.26
>>> calculate_bill(2.00, 0.13)
2.26
>>> calculate_bill(30.00, 0.05)
31.5
```

Memory model (step 2)

format_name.ipynb

Following the Function Design Recipe, write the following function.

Function name: (parameter types) → return type	Description
`(str, str) str`	The first parameter represents a first name and the second represents a last name. Return a string in the format `last_name, first_name`, where `last_name` and `first_name` are replaced by the given last and first names.

In [ ]

```
def format_name(first_name: str, last_name: str) -> str:
    """Return a string in the format last_name, first_name.

    >>> format_name('Michelle', 'Craig')
    'Craig, Michelle'
    >>> format_name('Jen', 'Campbell')
    'Campbell, Jen'
    """
    return last_name + ', ' + first_name
```

In [2]

```
format_name('Michelle', 'Craig')
```

```
'Craig, Michelle'
```

to_listing.ipynb

Following the Function Design Recipe, write the following function to produce a telephone directory listing. In the function body, **call** `format_name` to have it do some of your work for you.

Function name: (parameter types) → return type	Description
`format_name: (str, str, str) → str`	The first parameter represents a first name, the second represents a last name, and the third represents a phone number. Return a string in the format `last_name, first_name: phone_number`, where `last_name`, `first_name`, and `phone_number` are replaced by the given last name, first name, and phone number.

In [ ]

```
def format_name(first_name: str, last_name: str) -> str:
    """Return a string in the format last_name, first_name.

    >>> format_name('Michelle', 'Craig')
    'Craig, Michelle'
    """
    return last_name + ', ' + first_name
```

In [ ]

```
def to_listing(first_name: str, last_name: str, phone_number: str) -> str:
    """Return a string in the format last_name, first_name: phone_number.

    >>> to_listing('Michelle', 'Craig', '416-555-1234')
    'Craig, Michelle: 416-555-1234'
    """
    return format_name(first_name, last_name) + ': ' + phone_number
```

In [3]

```
to_listing('Michelle', 'Craig', '416-555-1234')
```

```
'Craig, Michelle: 416-555-1234'
```

A student wrote this body for `format_name`. What does `format_name('Michelle', 'Craig')` return?

```
1  def format_name(first_name: str, last_name: str) -> str:
2      return 'last_name' + ', ' + 'first_name'
```

A `'Craig, Michelle'`

B `'Michelle, Craig'`

✓ `'last_name, first_name'`

D `NameError`